Monitoring

Version: 1.0

Purpose

This document defines the monitoring strategy of Trading Platform Pro and its primary application, the Trading Cockpit.
Monitoring provides continuous visibility into:
• runtime health
• infrastructure capability state
• broker connectivity
• market data connectivity
• market data subscriptions
• persistence availability
• background services
• event processing
• reconciliation state
• operational performance
Monitoring observes application operation.
Monitoring shall never make trading decisions or silently change business state.

Objectives

The monitoring system shall provide:
• runtime visibility
• capability health visibility
• broker state visibility
• market data state visibility
• subscription state visibility
• persistence visibility
• background task visibility
• event processing visibility
• reconciliation visibility
• performance metrics
• early problem detection
• operational diagnostics
• historical analysis where required

Monitoring Principles

Monitoring shall be:
• observable
• lightweight
• deterministic
• non-intrusive
• capability-oriented
• operationally meaningful
• testable
Monitoring shall not:
• submit orders
• cancel orders
• retry order submissions
• reconnect external systems independently
• repair business state

• change Domain state
• change Application workflow state
Monitoring observes state.
Operational recovery belongs to explicitly owned runtime or application workflows.

Monitoring and Business Behaviour

Monitoring shall remain separate from business behaviour.
Example:
```text
Monitoring
↓
BROKER_DISCONNECTED observed
```
Monitoring may:
• record the state
• update health state
• expose operational impact
• raise an alert
Monitoring shall not automatically:
```text
Reconnect broker
Resubmit order
Cancel order
Close position
Repair local state
```
Recovery actions require explicit lifecycle or application ownership.

Monitoring Scope

Initial monitored capabilities include:
• Runtime
• Persistence
• Broker Integration
• Market Data Integration
• Market Data Subscriptions
• Background Tasks
• Scheduler
• Event Processing
• Messaging where implemented
• Reconciliation Workflows
• Presentation Runtime where operationally relevant
Only implemented capabilities shall be monitored.
Do not create monitoring infrastructure for hypothetical future services.

Capability State

Monitoring shall expose capability state explicitly.
A monitored capability may report:
• Healthy
• Degraded
• Unhealthy
• Unknown
Capability-specific operational state may additionally be exposed.
Example:

```text
Runtime
Health: Healthy
State: Operational
Broker
Health: Degraded
State: Disconnected
Market Data
Health: Healthy
State: Connected
```

Health and operational state are related but not identical.

Health State Definitions

Healthy

The capability is available and operating within defined expectations.

Degraded

The capability remains partially available or the application remains usable with reduced functionality.

Examples:

- broker disconnected while the Trading Cockpit remains usable
- one market data subscription failed
- non-critical background worker unavailable

Unhealthy

The capability cannot provide its required technical function.

Examples:

- required persistence unavailable
- market data adapter failed completely
- runtime service repeatedly fails

Unknown

The capability state cannot currently be determined.

Unknown shall not silently be treated as Healthy.

Runtime Monitoring

Runtime monitoring shall observe:

- runtime state
- startup phase
- shutdown phase
- degraded state
- critical failures
- runtime duration

Relevant runtime states may include:

- Bootstrapping
- Initializing
- Starting
- Operational

• Degraded
• Stopping
• Stopped
• Failed
Runtime state transitions shall be observable.

Startup Monitoring

Startup monitoring may measure:
• configuration duration
• dependency construction duration
• persistence initialization duration
• broker initialization duration
• market data initialization duration
• application service initialization duration
• presentation initialization duration
• workspace restoration duration
• total startup duration
Startup monitoring shall not alter startup order.

Persistence Monitoring

Persistence monitoring shall observe:
• database availability
• connection readiness
• transaction failures
• repository failures
• schema readiness where relevant
Persistence health checks shall remain lightweight.
Health checks shall not mutate business data.
Do not use write operations solely to determine database health unless explicitly required and safely designed.

Broker Monitoring

Broker monitoring shall observe:
• broker provider
• PAPER or LIVE environment
• connection state
• connection duration
• disconnect events
• reconnect state
• connection failures
• broker adapter failures
Broker monitoring shall not initiate broker reconnection.
Broker reconnection belongs to the Broker Connection Supervisor or another explicitly owned runtime component.

Broker Health

Broker health may consider:
• adapter availability
• connection state
• connection failure
• reconnect state

Example mapping:
```text
Connected → Healthy
Reconnecting → Degraded
Disconnected → Degraded
Failed → Unhealthy
Unknown → Unknown
```
The exact mapping shall be defined by operational requirements.
Broker health shall remain distinguishable from overall runtime health.

PAPER and LIVE Monitoring

Trading-related monitoring shall preserve execution environment context.
Use explicit values:
```text
PAPER
LIVE
```
LIVE capability degradation shall remain clearly visible.
Monitoring shall not infer execution environment from broker port or account naming conventions.
The active environment shall come from validated application configuration.

Market Data Monitoring

Market data monitoring shall observe:
• provider
• connection state
• connection failures
• subscription capability
• freshness state
• provider errors
• stale data conditions
Market data state shall remain distinguishable from broker state.
The broker may be connected while market data is unavailable.

Market Data Health

Market data health may consider:
• provider connection
• quote availability
• freshness state
• subscription failures
Example mapping:
```text
Connected and current → Healthy
Connected but stale → Degraded
Disconnected → Degraded
Provider failed → Unhealthy
State unavailable → Unknown
```
Freshness thresholds that affect trading workflows require explicit Application or product ownership.
Monitoring may observe freshness state.
Monitoring shall not define trading decisions based on freshness.

Subscription Monitoring

Market data subscription monitoring shall observe:
• requested subscriptions
• active subscriptions
• stale subscriptions
• failed subscriptions
• closing subscriptions
• closed subscriptions
Relevant metrics may include:
• active subscription count
• failed subscription count
• stale subscription count
• subscription activation duration
Monitoring shall not create or close subscriptions independently.
Subscription lifecycle belongs to the owning application or runtime component.

Background Task Monitoring

Background task monitoring shall observe tasks registered with the Task Supervisor.
Relevant state may include:
• Starting
• Running
• Cancelling
• Stopped
• Failed
Metrics may include:
• active task count
• failed task count
• task runtime
• cancellation duration
Unmanaged background tasks cannot be reliably monitored.
Long-running application tasks shall therefore have explicit lifecycle ownership.

Scheduler Monitoring

Scheduler monitoring may observe:
• scheduler state
• scheduled task count
• execution latency
• failed scheduled callbacks
• delayed execution
Monitoring shall not define scheduling rules.
Trading workflow timing belongs to Application or Domain ownership where business-relevant.

Event Processing Monitoring

Event processing monitoring may observe:
• dispatcher state
• event throughput
• queue depth
• handler failures
• processing duration
• delayed events
Critical event handler failures shall remain visible.

Monitoring shall not replay business-critical events automatically unless an explicitly designed event recovery workflow owns replay behaviour.

Messaging Monitoring

Where messaging is implemented, monitoring may observe:
• queue depth
• delivery failures
• processing latency
• consumer state
Messaging monitoring shall not silently discard or recreate business-critical messages.
Delivery recovery requires explicit messaging infrastructure policy.

Reconciliation Monitoring

Reconciliation monitoring is operationally important.
Monitoring may observe:
• reconciliation cycle state
• cycle duration
• discrepancy count
• discrepancy classification
• unresolved discrepancy count
• reconciliation failures
Relevant states may include:
• Idle
• Running
• Action Required
• Completed
• Failed
Monitoring shall not resolve discrepancies.
Reconciliation decisions belong to Application workflows and Domain rules.

Reconciliation Health

Reconciliation health may consider unresolved discrepancies.
Example:
```text
No unresolved discrepancy
→ Healthy
Known discrepancy under review
→ Degraded
Reconciliation unavailable
→ Unhealthy
```
A discrepancy shall not automatically make the complete runtime unhealthy.
Operational impact shall remain capability-specific.

Presentation Runtime Monitoring

Presentation monitoring may observe operationally relevant state such as:
• main window initialization
• workspace restoration failure
• critical widget initialization failure
• UI event loop responsiveness
Monitoring shall not record trivial user interactions.

Do not monitor:
• mouse movement
• every selection change
• every repaint
• every button hover
Trading actions shall be observed through Application workflows.

UI Responsiveness

The Trading Cockpit shall remain responsive.
Monitoring may detect:
• long UI thread blocking
• delayed event processing
• long-running synchronous callbacks
UI responsiveness metrics shall have minimal overhead.
Monitoring shall not modify UI workflow behaviour to hide performance problems.

AsyncIO Monitoring

AsyncIO monitoring may observe:
• event loop lag
• task count
• failed tasks
• long-running callbacks
• cancellation duration
Event loop monitoring shall remain lightweight.
Monitoring shall not create competing event loops.

Event Loop Lag

Event loop lag may indicate:
• blocking operations
• CPU-intensive synchronous work
• uncontrolled logging
• long callbacks
Lag thresholds shall be defined operationally.
Monitoring may report:
```text
EVENT_LOOP_LAG_DETECTED
```
Monitoring shall not automatically terminate tasks based solely on lag detection.

Metrics

Metrics shall represent measurable operational state.
Initial metric categories may include:
• runtime
• persistence
• broker
• market data
• subscriptions
• background tasks
• scheduler
• event processing
• reconciliation

• UI responsiveness
Only useful metrics shall be collected.
Avoid metrics without an operational consumer.

Runtime Metrics

Runtime metrics may include:
• application uptime
• startup duration
• shutdown duration
• degraded duration
• runtime state transition count

Persistence Metrics

Persistence metrics may include:
• operation duration
• transaction failure count
• repository failure count
High-cardinality business identifiers shall not automatically become metric labels.

Broker Metrics

Broker metrics may include:
• connection state
• connection duration
• disconnect count
• reconnect attempt count
• connection failure count
Order state belongs primarily to business and operational workflow observability.
Avoid uncontrolled per-order metric labels.

Market Data Metrics

Market data metrics may include:
• connection state
• active subscription count
• stale subscription count
• failed subscription count
• subscription activation duration
High-frequency quote values are not monitoring metrics by default.

Task Metrics

Task metrics may include:
• active task count
• failed task count
• task duration
• cancellation duration
Task identity shall avoid uncontrolled high-cardinality metric labels.

Event Metrics

Event metrics may include:
• event throughput

• queue depth
• handler failure count
• processing duration
Domain event payload values shall not automatically become metric labels.

Reconciliation Metrics

Reconciliation metrics may include:
• reconciliation cycle duration
• discrepancy count
• unresolved discrepancy count
• reconciliation failure count
Discrepancy identity belongs in structured logs or persisted reconciliation state rather than high-cardinality metrics.

Metric Cardinality

Metric labels shall have controlled cardinality.
Avoid labels such as:
• order_id
• position_id
• candidate_id
• decision_id
• execution_id
for general metrics.
Use structured logs for identity-specific investigation.
Metrics support aggregation and trend analysis.
Logs support event-level investigation.

Logging Integration

Monitoring complements logging.
Logging answers:
> What happened?
Monitoring answers:
> What is the current operational state and how is it changing?
Examples:
```text
Logging:
BROKER_DISCONNECTED
Monitoring:
broker_health=Degraded
broker_state=Disconnected
```
Monitoring and logging shall use consistent capability terminology.

Health Checks

Health checks evaluate current technical capability availability.
Health checks may include:
• runtime
• persistence
• broker adapter
• market data adapter
• task supervisor

• event dispatcher
Health checks shall:
• execute quickly
• avoid business state mutation
• avoid order submission
• avoid order cancellation
• avoid reconciliation repair
• avoid expensive full-system scans

Health Check Timeouts

External health checks shall use explicit timeout behaviour where appropriate.
A health-check timeout shall result in:
```text
Unknown
```
or:
```text
Unhealthy
```
according to the defined capability policy.
Timeout shall not silently result in Healthy.

Health Check Failures

A health check failure shall identify:
• capability
• resulting health state
• failure category
• timestamp
Unexpected technical exceptions shall be logged.
Repeated health check failures may trigger alerts.
Monitoring shall not repair the failing capability.

Alerting

Alerts communicate operational conditions requiring attention.
Potential alert conditions include:
• runtime failed
• required persistence unavailable
• repeated broker connection failure
• LIVE broker disconnected
• market data unavailable
• repeated subscription failures
• event loop lag
• background worker failure
• reconciliation action required
Alerting shall remain separate from business decision logic.

Alert Severity

Alert severity may include:
• Info
• Warning
• Critical

Severity shall represent operational impact.
Example:
```text
PAPER broker disconnected
→ Warning
LIVE broker disconnected
→ Critical or high-priority Warning
```
Exact severity policy shall be defined operationally.

Alert Context

Alerts shall contain enough context for diagnosis.
Relevant context may include:
• capability
• health state
• operational state
• PAPER or LIVE environment
• first observed timestamp
• latest observed timestamp
• occurrence count
Sensitive information shall not be included.

Alert Deduplication

Repeated identical conditions shall not create uncontrolled alert floods.
Alerting may support:
• deduplication
• suppression windows
• state-based alert transitions
Example:
```text
Healthy
↓
Degraded
→ Alert
Degraded
↓
Degraded
→ No duplicate alert
Degraded
↓
Healthy
→ Recovery notification
```
Alert deduplication shall not hide new failure categories.

Operational Dashboard

Monitoring infrastructure should support a Trading Cockpit operational view.
Potential dashboard sections:
• Runtime
• Trading Environment
• Broker
• Market Data
• Subscriptions

• Persistence
• Background Tasks
• Event Processing
• Reconciliation
The dashboard shall expose capability state without requiring users to inspect raw logs.

Dashboard State

Example:
```text
Runtime Healthy Operational
Environment LIVE
Broker Degraded Reconnecting
Market Data Healthy Connected
Subscriptions Healthy 24 Active
Persistence Healthy Available
Reconciliation Degraded Action Required
```
Operational state shall remain concise and understandable.

Historical Monitoring

Historical monitoring may support:
• incident analysis
• trend analysis
• performance regression detection
• capacity evaluation
Historical monitoring shall only be introduced where an operational consumer exists.
Do not persist unlimited metrics by default.
Retention shall be configurable.

Monitoring Performance

Monitoring shall have minimal runtime impact.
Monitoring shall:
• avoid blocking the UI thread
• avoid blocking the AsyncIO event loop
• avoid excessive allocations
• avoid uncontrolled polling
• avoid high-frequency persistence writes
Monitoring itself shall be monitored where operationally justified.

Monitoring Failure

Monitoring failure shall remain distinguishable from application capability failure.
Example:
```text
Broker operational
Monitoring collector failed
```
The application shall not mark the broker unhealthy solely because the monitoring collector failed.
Monitoring failure shall be logged and exposed where relevant.

Testing

Monitoring components shall be independently testable.
Automated tests shall verify where relevant:
• health state mapping
• degraded state
• unknown state
• broker health mapping
• market data health mapping
• subscription monitoring
• task failure monitoring
• event loop lag detection
• reconciliation monitoring
• health check timeout
• alert severity
• alert deduplication
• recovery notification
• metric cardinality rules
• PAPER and LIVE context
• monitoring does not mutate business state
Tests shall not require LIVE trading.

Monitoring Evolution

Monitoring evolves with implemented operational capabilities.
Before introducing monitoring for a new capability:
1. Identify the operational consumer.
2. Identify the monitored capability.
3. Define operational state.
4. Define health mapping.
5. Define useful metrics.
6. Define alert conditions.
7. Evaluate metric cardinality.
8. Evaluate runtime overhead.
9. Add automated tests.
10. Update documentation.
Avoid monitoring created only for hypothetical future scalability.

Monitoring Review Checklist

Before introducing or changing monitoring verify:
• operational consumer identified
• monitored capability identified
• health states defined
• operational states defined
• health mapping defined
• PAPER and LIVE impact evaluated
• broker monitoring does not reconnect
• market data monitoring does not control subscriptions
• monitoring does not submit orders
• monitoring does not cancel orders
• monitoring does not repair reconciliation state
• health checks remain lightweight
• timeout behaviour defined
• useful metrics identified
• metric cardinality controlled
• alert condition defined
• alert severity defined

• alert deduplication evaluated
• runtime overhead evaluated
• automated tests added
• documentation synchronized

Related Documents

• Product_Vision.md
• Product_Roadmap.md
• Project_Overview.md
• Architecture.md
• Infrastructure.md
• Technical_Specifications.md
• Configuration.md
• Runtime.md
• Logging.md
• API_Guidelines.md
• Testing_Strategy.md
• AGENTS.md